마이칩 5주차

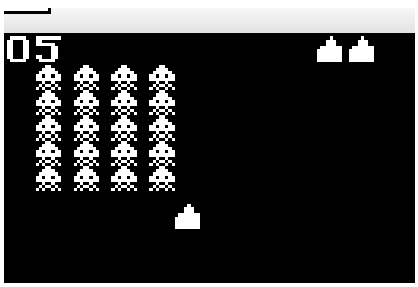
2022012330 이현서

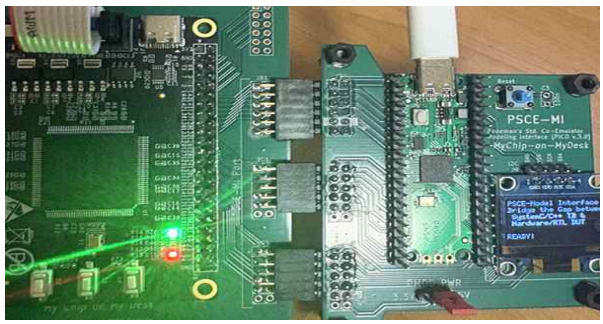
Q13

4_space_invaders_engine_glcd.v.Ok를 검증 대상으로 선택하고, 소프트 링크가 올바르게 설정되었는지 확인하였다. 이후 make lint로 RTL을 검사하고, make build로 시뮬레이터를 생성하였다. make run으로 게임의 동작을 확인하고 make wave로 파형을 관찰하는 과정까지 성공적으로 수행하였다.
다음으로 FPGA 에뮬레이션 검증을 위해 emulation 디렉토리로 이동하였다.

먼저 make build_mi로 Raspberry Pi Pico의 펌웨어를 빌드하고, BOOTSEL 모드에서 드라이브를 WSL에 마운트한 뒤 make upload_mi로 업로드했다. 이후 FPGA용 회로를 빌드하고, Digilent USB 장치를 WSL에 연결하여 make config_trans를 수행했다. 마지막으로 테스트벤치를 빌드하고 make run으로 게임 실행을 시도했다.
발생한 문제
처음에는 Pico의 업로드용 드라이브가 WSL에서 인식되지 않아 No drive to deploy 오류가 발생했지만, 마운트 후 업로드에 성공했다. FPGA 설정 과정에서도 USB 장치가 연결되지 않아 오류가 발생했으나 usbipd attach로 해결했다. 실행 단계에서는 시리얼 포트를 찾지 못하거나 Connection failed가 발생했다. 중간에 연결되어 게임이 표시되기도 했지만 속도가 매우 느렸고, 다시 동일 과정을 실행했을땐 보드에 아무것도 나오지 않으며 업로드에 실패으며, . 최종적으로 안정적인 실행을 확인하지 못했다.

emulation





과제 8 : 포대가 급격히 이동했던 건 사람이 키를 누르는 시간과 수백만 Hz로 도는 고속 시스템 클럭 사이의 속도 차이 때문이었다. 버튼이 'High' 상태로 유지되는 동안 매 클럭마다 좌표가 계속 누적돼서 생기는 문제라, 버튼 입력 처리를 화면 갱신 주기에 맞춰 동기화해야 한다. 이를 해결하려면 버튼 신호에 디바운싱이랑 상승 엣지 감지 회로를 넣어서 딱 처음 누른 순간에만 반응하게 만들면 된다. 추가로 입력 감지랑 좌표 갱신 로직을 분리해서 화면의 프레임 단위로 한 프레임당 일정 픽셀씩만 부드럽게 이동하게 제한하면, 과도한 이동을 막고 안정적인 움직임을 구현할 수 있을 것이다.

과제 9 : 비디오 신호에서 V_Sync는 한 프레임의 출력이 끝나고 다음 프레임이 시작되는 경계 지점을 알려주는 필수적인 신호이다. 이 신호가 존재해야 디스플레이가 정확한 프레임 단위로 화면을 그릴 수 있으며, 그렇지 않으면 프레임 경계를 찾지 못해 화면이 위아래로 흐르거나 찢어지는 현상이 발생한다. 또한 V_Sync 구간을 통해 화면 버퍼를 갱신하거나 빔이 제자리로 돌아갈 시간을 확보할 수 있어, 안정적인 60Hz 주사율과 수직적 화면 안정성을 보장할 수 있다. 따라서 V_Sync는 영상의 왜곡 없이 매끄럽고 안정적인 출력을 구현하기 위해 반드시 필요해보인다.

과제 10 : FPGA 에뮬레이션을 거쳐야 하는 이유는 실제 물리적인 버튼 입력의 비동기성과 채터링 현상, 그리고 실제 디스플레이 장치와의 정밀한 타이밍 인터페이스를 직접 검증할 수 있기 때문이다. 또한 시뮬레이션은 수많은 클럭을 처리하는 데 오랜 시간이 소요되지만, FPGA는 실제 하드웨어 속도로 실시간 구동되므로 실제 환경에서 발생할 수 있는 예외 상황과 성능 한계를 정확하게 확인하기 위해 필수적이다.

과제 11 :인공지능이 지적하듯 하드웨어 에뮬레이션은 내부 신호 추적이 어렵다는 한계가 있지만, 시놉시스의 ZeBu나 케이던스 Palladium 같은 상용 하드웨어 가속기들은 이를 극복하기 위해 다양한 대책을 마련해 두고 있다. 우선 에뮬레이터 내부에 대용량의 전용 트레이스 메모리를 탑재하여 실행 중인 주요 내부 신호의 상태를 실시간으로 저장함으로써 추후 시뮬레이터처럼 파형 분석이 가능하도록 지원한다. 또한 과거와 달리 전체 디자인을 처음부터 다시 컴파일할 필요 없이 사용자가 원하는 신호나 포인트를 실시간으로 선택해 모니터링할 수 있는 런타임 프로빙 기능을 제공한다. 여기에 전용 디버깅 툴과의 긴밀한 연동을 더해 소프트웨어 환경 못지않은 가시성을 확보함으로써 하드웨어 에뮬레이션의 디버깅 효율성을 높이고 있다.

과제 12 : (a) 쿼터스 합성 스크립트에서 확인한 FPGA 부품명은 EP4CE6E22C8(Cyclone IV E 패밀리, TQFP 144핀 패키지, 스피드 등급 8)입니다. 이 스크립트의 핀 할당 내용과 PSCE-TRANS 보드의 입출력을 비교해보면, 시스템 클록(clk: PIN_135, clk_o: PIN_119)과 리셋(rst_n: PIN_114), 조작 버튼(btn_left: PIN_112, btn_right: PIN_24, btn_fire: PIN_111)이 각각 지정되어 있으며, LCD 제어 신호(lcd_write_en: PIN_120, lcd_data: PIN_121)와 X/Y 좌표 버스(lcd_x[6:0], lcd_y[5:0])가 하드웨어 사양에 맞게 정확히 매핑되어 있습니다.

(b) 프로토타이핑 펌웨어와 에뮬레이션 모델링 인터페이스 펌웨어를 비교하면, 먼저 통신 및 제어 방식에서 프로토타이핑 펌웨어는 RP2040의 개별 GPIO 핀을 직접 digitalRead/digitalWrite로 제어하는 반면, 에뮬레이션 펌웨어는 PSCE 프레임워크 기반의 패킷 통신 방식(psce.RxPacket 및 psce.TxPacket)을 사용합니다. 데이터 처리 방식에 있어서도 프로토타이핑 펌웨어는 클록 인터럽트 시점에 개별 핀 상태를 일일이 읽어오지만, 에뮬레이션 펌웨어는 수신된 패킷 바이트를 비트 연산으로 디코딩하여 처리합니다. 결과적으로 프로토타이핑 펌웨어는 독립된 하드웨어 환경에서 실시간 동작을 검증하는 데 초점이 맞춰져 있고, 에뮬레이션 펌웨어는 표준화된 인터페이스를 통해 코에뮬레이션과 모델링을 효율적으로

수행하도록 구조화되어 있습니다.

과제 13 : 보드에 게임을 구현하지 못하여 오실로스코프 측정하지 못함